Block 5 – Session 1

Software-Defined & Cloud-Native Networking

Arquitectura de Xarxes

Universitat Pompeu Fabra

upf.

Outline

upf.

What If You Could Program Your Entire Network?

Traditional networks are configured device by device.
What if the network was as programmable as software?

Learning objectives:

1. Explain SDN architecture and why it matters
2. Understand NFV and how it replaces hardware appliances
3. Describe container networking and microservices communication

Traditional Network Management

- Each device (router, switch, firewall) configured **individually**
- Configuration via CLI, **device by device**
- Vendor-specific commands and interfaces
- Changes require **manual intervention** on every device

Imagine updating 1,000 switches one by one. That's traditional networking.

Scalability Challenges

- Data centers grow from hundreds to **hundreds of thousands** of devices
- Manual configuration does **not scale**
- Network changes are **slow**: days or weeks for approval + implementation
- **Human errors** increase with complexity
- Cloud-scale infrastructure demands **automation**

The Control Plane Problem

- In traditional networks, **every device** runs its own control plane
- Each router independently computes routes (OSPF, BGP from Block 3)
- No **centralized view** of the entire network
- Difficult to implement **network-wide policies**
- Limited **programmability**: cannot easily add new features

The control plane is **distributed** by design. What if we **centralized** it?

Discussion: The Cost of Manual Networks

Why have traditional networks survived so long despite their scalability problems?

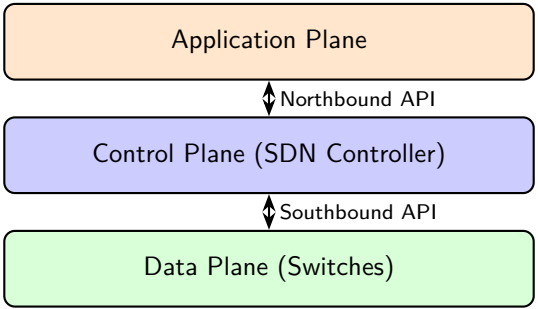
- Hint: think about reliability, vendor relationships, and risk aversion
- What would it take for an organization to change?

Outline

upf.

SDN – Core Idea

- **Separate** the control plane from the data plane
- **Centralize** network intelligence in a software controller
- **Program** network behavior through APIs



McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM CCR*, 2008.

SDN Architecture – Three Planes

Data Plane (Infrastructure Layer):

- Physical/virtual switches that **forward packets**
- No longer make independent routing decisions
- Receive forwarding rules from the controller

Control Plane (Controller):

- Centralized software with a **global view** of the network
- Computes paths, installs forwarding rules on switches
- Single point of management for the entire network

Application Plane:

- Applications that define **network behavior** (firewall, load balancer, monitor)
- Communicate with the controller via northbound APIs

upf.

SDN Controllers

- The controller is the **brain** of an SDN network
- Maintains a real-time **topology database** of the entire network
- Supports **high availability** (clustered deployment)

Examples:

- **OpenDaylight** (Linux Foundation, Java-based, widely adopted)
- **ONOS** (Open Network Operating System, carrier-grade, telecom-focused)
- **Floodlight** (open-source, lightweight, educational)
- Proprietary: **Cisco ACI**, **VMware NSX**

OpenDaylight Project, opendaylight.org; ONOS Project, onosproject.org.

upf.

Southbound Interface – OpenFlow

- **OpenFlow** (2008, Stanford): the original SDN protocol
- Defines how the controller communicates with switches
- Controller installs **flow rules** in the switch's flow table:

Match	Action	Priority
dst IP = `10.0.1.0/24`	Forward to port 3	100
dst IP = `10.0.2.0/24`	Forward to port 5	100
src IP = `192.168.1.100`	Drop	200
* (any)	Send to controller	1

- If no rule matches → packet sent to controller for a decision

ONF, "Software-Defined Networking: The New Norm for Networks," ONF White Paper, 2012.

Northbound Interface – Representational State Transfer (REST) APIs

- Applications interact with the controller via **REST APIs**
- Example operations:
  - `GET /topology/links` → get network topology
  - `POST /flows` → add a flow rule
  - `GET /statistics/port` → query traffic statistics

Key benefit: the network becomes **programmable** like any other software system.

Any developer can write applications that control network behavior.

upf.

SDN Benefits – Summary

- **Centralized management**: one controller, global network view
- **Programmability**: automate changes via APIs
- **Agility**: deploy new policies in seconds, not days
- **Vendor independence**: OpenFlow works across vendors
- **Innovation**: researchers and developers can experiment easily

SDN is the foundation for modern cloud networking: AWS, Azure, and GCP all use SDN internally.

Discussion: SDN Trade-offs

If the SDN controller fails,
what happens to the network?

- Hint: single point of failure vs distributed control
- How do real deployments address this? (clustering, failover)

Outline

upf.

From Hardware Appliances to Software

- Traditional network functions run on **dedicated hardware**:
  - Firewall → physical appliance ($10K–$100K+)
  - Load balancer → physical appliance
  - Router → physical appliance
- Problems: **expensive**, inflexible, vendor lock-in, slow to deploy
- NFV idea: run these functions as **software on commodity servers**

ETSI, "Network Functions Virtualisation – An Introduction, Benefits, Enablers, Challenges & Call for Action," White Paper, 2012.

Virtual Network Functions (VNFs)

- A **VNF** = network function implemented as software
- Runs on VMs or containers, on standard x86 servers

Physical Appliance	**VNF Equivalent**
Hardware firewall	Virtual firewall (pfSense, iptables)
Hardware load balancer	Virtual LB (HAProxy, NGINX)
Hardware router	Virtual router (VyOS, FRRouting)
WAN optimizer	Virtual WAN optimizer
Intrusion Detection / Prevention (IDS/IPS)	Virtual IDS (Snort, Suricata)

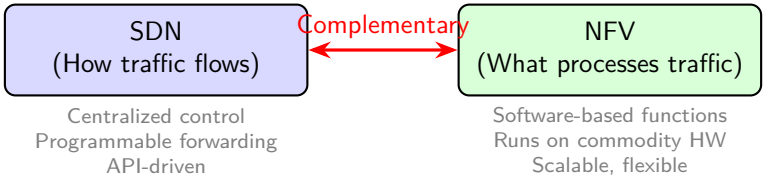
ETSI GS NFV 002, "NFV Architectural Framework," v1.2.1, 2014.

NFV + Cloud Integration

- Deploy VNFs on cloud infrastructure (IaaS) $\rightarrow$ **minutes** instead of months
- **Scale horizontally**: add more instances under load
- **Update easily**: software upgrades, no truck rolls
- **Cost reduction**: commodity servers vs specialized hardware

No hardware procurement, no shipping, no rack-and-stack.

SDN + NFV – Complementary Technologies



- SDN **steers** traffic to the right VNF (forwarding rules)
- NFV **processes** the traffic (filter, balance, encrypt, inspect)

Discussion: When Hardware Still Wins

Can you think of a scenario where a physical appliance is better than a VNF?

- Hint: think about latency, throughput, and specialized workloads
- What about hardware accelerators (FPGAs, SmartNICs)?

Outline

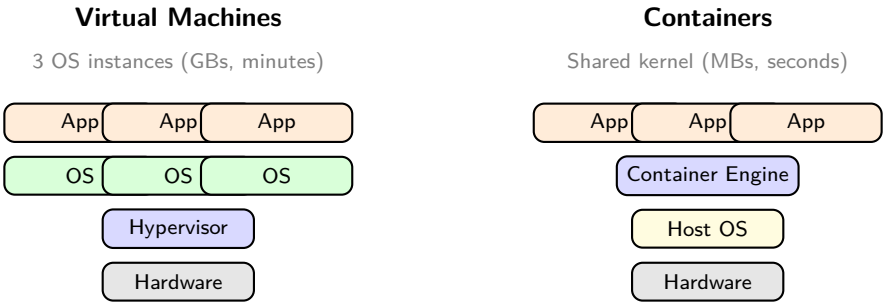
upf.

Containers – Deep Dive (Expanding Block 4 Overview)

- In Block 4 we compared VMs vs containers at a high level
- Now: **how containers actually work**
- Containers share the **host kernel** (unlike VMs with full guest OS)
- Isolation via Linux kernel features:
  - **Namespaces**: isolate process trees, network stack, filesystem
  - **cgroups**: limit CPU, memory, I/O per container
- **Docker**: most popular container platform
  - Packages app + dependencies into a **container image**
  - Image runs as a **container**: lightweight, portable, reproducible

Docker, Inc., "Docker Overview," docs.docker.com; OCI, "Open Container Initiative," opencontainers.org.

VM vs Container – Revisited



- VMs: **minutes** to start, **GBs** in size, 3 full OS running
- Containers: **milliseconds** to start, **MBs** in size, shared kernel
- In practice: containers often run **inside** VMs for extra isolation

Container Network Models

Bridge network (default):

- Containers on the same host connected via a **virtual bridge** (`docker0`)
- Each container gets a **veth pair** (virtual Ethernet interface)
- **NAT** for external traffic; containers share host's IP

Host network:

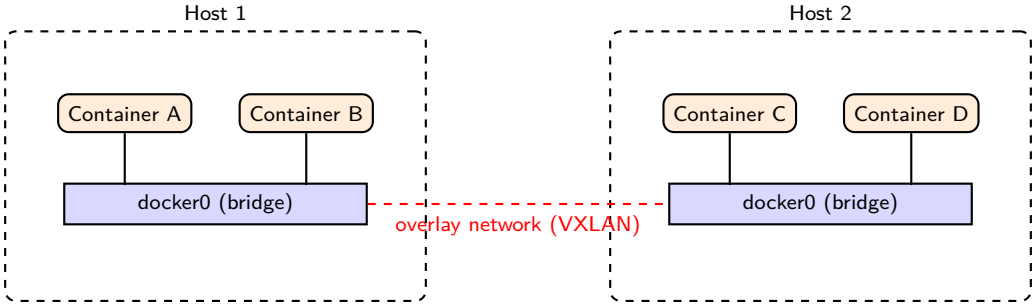
- Container uses the **host's network stack directly**
- No network isolation, but **no NAT overhead**
- Use for performance-critical applications

Overlay network:

- Containers on **different hosts** connected via VXLAN overlay
- Same overlay concept from Block 4, now applied to containers
- Enables **multi-host** container deployments

Docker, Inc., "Docker Networking Documentation," docs.docker.com/network.

Container Networking – Visual



- Within a host: containers use a **bridge** network
- Across hosts: an **overlay** connects the bridges (same VXLAN from B4)

Microservices – From Monolith to Distributed

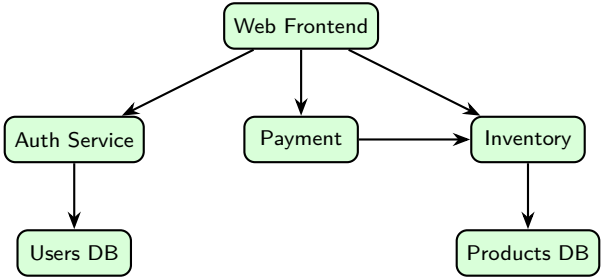
- **Monolithic** app: one big process, one deployment, one codebase
- **Microservices**: app split into small, independent services
  - Each service runs in its **own container**
  - Services communicate over the **network** (HTTP/REST, gRPC Remote Procedure Call)

Network implications:

- Hundreds of containers talking to each other
- Need **service discovery**: "where is the payment service?"
- Need **load balancing**: distribute requests across replicas
- Need **observability**: trace requests across services

upf.

Microservices – Network Challenges



- Every arrow = **network call** → latency and failure risk
- If Inventory is down → Payment and Web are affected (**cascading failure**)
- Solutions: retries, timeouts, circuit breakers, service meshes

Discussion: Containers vs VMs

If containers are faster and lighter than VMs,
why do companies still use VMs?

- Hint: security isolation, legacy applications, compliance requirements
- In practice, containers often run inside VMs

Outline

upf.

Infrastructure as Code (IaC)

- **IaC**: manage infrastructure through **code files**, not manual clicks
- Describe desired state in a configuration file → tools apply it automatically

Key benefits:

- **Reproducibility**: same config → same infrastructure, every time
- **Version control**: track changes in Git, review, rollback
- **Automation**: no manual steps → fewer human errors
- **Speed**: deploy entire environments in minutes

Morris, *Infrastructure as Code*, 2nd ed., O'Reilly, 2021.

IaC Example – Terraform (Conceptual)

```
# Define a VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# Define a subnet
resource "aws_subnet" "public" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
}

# Define a security group
resource "aws_security_group" "web" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 80
```

Discussion: IaC Risks

If infrastructure is defined as code,
what happens when someone pushes a bug?

- Hint: code review, staging environments, automated testing
- How is this similar to software development best practices?

Outline

upf.

Serverless Computing (FaaS)

- **Serverless** (Function as a Service, FaaS):
  - Deploy individual **functions**, not entire servers
  - Triggered by **events** (HTTP request, file upload, timer)
  - Provider manages **all infrastructure**: zero server management
  - Pay only for **execution time** (measured in milliseconds)
- Examples: **AWS Lambda**, **Azure Functions**, **Google Cloud Functions**

Trade-offs: less control, potential vendor lock-in, cold start latency.

AWS, "AWS Lambda Documentation," docs.aws.amazon.com/lambda.

upf.

Edge Computing

- **Edge computing**: process data **close to where it is generated**
- Instead of sending everything to a central cloud → process at the edge
- Benefits:
  - **Lower latency** (critical for IoT, autonomous vehicles, AR/VR, gaming)
  - **Less bandwidth** (filter and aggregate data locally)
  - **Better privacy** (data stays closer to the source)

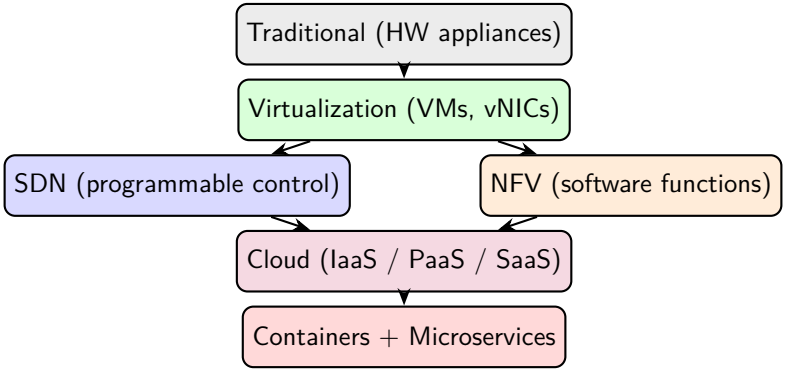
Cloud and edge are **complementary**, not competing:

- Edge: real-time, latency-sensitive tasks
- Cloud: heavy computation, long-term storage, analytics

ETSI, "Multi-access Edge Computing (MEC)," etsi.org/technologies/multi-access-edge-computing.

upf.

The Full Picture – Where Everything Fits



Each layer builds on the previous one. This is the **evolution of networking**.

References

1. McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM CCR*, vol. 38, no. 2, 2008.
2. ONF, "Software-Defined Networking: The New Norm for Networks," ONF White Paper, 2012.
3. OpenDaylight Project, opendaylight.org.
4. ONOS Project, onosproject.org.
5. ETSI, "Network Functions Virtualisation," White Paper, 2012.
6. ETSI GS NFV 002, "NFV Architectural Framework," v1.2.1, 2014.
7. Docker, Inc., "Docker Overview and Networking," docs.docker.com.
8. OCI, "Open Container Initiative Runtime and Image Specifications," opencontainers.org.
9. Morris, *Infrastructure as Code*, 2nd ed., O'Reilly, 2021.
10. HashiCorp, "Terraform Documentation," terraform.io/docs.
11. AWS, "AWS Lambda Documentation," docs.aws.amazon.com/lambda.
12. ETSI, "Multi-access Edge Computing (MEC)," etsi.org/technologies/multi-access-edge-computing.
13. Goransson & Black, *Software Defined Networks*, 2nd ed., Morgan Kaufmann, 2017.
14. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.

upf.

Outline

upf.

Key Takeaways

1. Traditional networks are **manual, rigid, and do not scale**
2. **SDN** separates control from data plane $\rightarrow$ centralized, programmable
3. **NFV** replaces hardware appliances with software $\rightarrow$ flexible, cheap
4. SDN + NFV are **complementary**: SDN steers, NFV processes
5. **Containers** are lightweight (shared kernel), networked via bridges/overlays
6. **Microservices** = many containers communicating over the network
7. **IaC** automates infrastructure $\rightarrow$ reproducible, fast, auditable

Discussion

A company runs 200 microservices in containers.
One Friday at 5 PM, a manual network change
breaks communication for 50 of them.
How could SDN and IaC have prevented this?

Hints: centralized control, automated testing, instant rollback, reproducibility.